Knapplan

Intelligent Course Planner

Design Report

CS 430 - Group 1
Noah Husby, Anna Gibes, Bolin Zhu
July 7th, 2026

Overview

Knapplan is an intelligent course planner that finds the classes to optimize studying for in order to get the best possible grade outcome. The name comes from the internal use of a “Knapsack” style dynamic programming algorithm. The software expects the input in the following form:

```
course1   hours1    benefit1
course2   hours2    benefit2
…
course(n) hours(n) benefit(n)
```

Figure 1: Input format

The software is a command line tool written in Python. It can be run on any system with a valid python3 runtime installed. By default, the tool will look for an *input.txt* file in the executing root of the program. An additional argument can be provided to specify a custom input file. Additionally, the hours and output files can be specified using -h and -o respectively. If these are not provided, the program will prompt you for that information in real time. These parameters can be used to automate the tool as needed.

The tool automatically validates the input to confirm that the input is correct. If the input is invalid or missing, the proper error messages will be displayed to the end user to clearly define the issue.

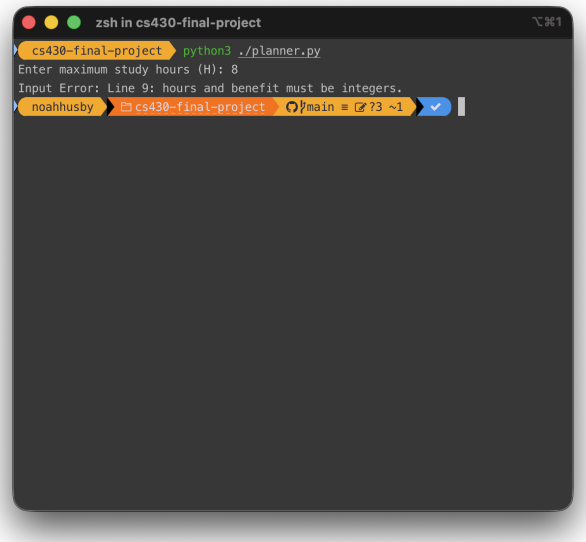

Figure 2: Error logging for invalid input

Algorithm

The project implemented a bottom-up dynamic programming solution to the 0/1 Knapsack problem. Each course was treated as a single item where the weight was the required study hours and the value was the benefit of taking the course. The objective is to maximize the total benefit while keeping the total study hours less than or equal to the maximum allowed hours.

DP State Definition

The planner uses a two-dimensional dynamic programming table named dp. The entry dp[i][h] uses only the first i courses from the input file. It then asks, what is the highest total benefit we can get without using more than h study hours? Here, i goes from 0 to n, where n is the number of courses, and h goes from 0 to H, where H is the maximum number of study hours entered by the user.

This state works for the course planner because every course has only two possible choices. The user either skips the course, or we take it if it fits inside the hour limit. The table does not need to store every previous schedule. It only needs the best outcome from earlier courses, because the next choice can be built from that smaller problem.

The base cases are simple. If no courses have been considered, the best benefit is 0. If the hour limit is 0, the best benefit is also 0, since every valid course has a positive number of study hours. This is why the first row and first column of the table start at 0.

For a course i, the program looks at courses[i - 1] because the course list is zero-indexed in Python. If that course takes more than h hours, it cannot be added, so the value stays dp[i -
1][h]. If it does fit, the program compares the two choices:

```
without_course = dp[i - 1][h]
with_course = dp[i - 1][h - course.hours] + course.benefit
dp[i][h] = max(without_course, with_course)
```

The final answer is stored in dp[n][H], which gives the best total benefit using all available courses and at most the maximum number of study hours.

The implementation also keeps two helper tables next to dp. The selected_count table is used only for tie breaking, this is when two choices give the same benefit. In that case, the program chooses the option with more selected courses. The keep_course

table records whether a course was chosen at a certain state. After the DP table is filled, the program walks backward through keep_course to recover the actual list of courses printed in the output.

Memoization Method

The planner does not use memoization because the algorithm is implemented using a bottom-up dynamic programming approach. Unlike a top-down approach, this solution does not use recursion or memoization to cache previously calculated results. Instead, the program builds the entire dp table iteratively by filling one row at a time. The algorithm starts with the base cases, and then works upward until every possible combination of courses and study hours has been evaluated. Since every subproblem is visited once in a fixed order, memoization is unnecessary.

Each entry in the dp table is calculated exactly once using values that have already been calculated from the previous row of the table. Once a result has been stored, it can be reused whenever it is needed without performing the same calculation multiple times. This allows the algorithm to avoid redundant work while building the optimal solution from previously computed results.

Time Complexity

The DP table contains (n+1)*(H+1) entries, where n is the number of courses and H is the maximum available study hours. The outer loop processes each course once, while the inner loop checks every possible study hour limit from 0 to H. For each table entry, the program determines whether the current course can be included. If the course fits, the algorithm compares the benefit of including the course against the benefit of excluding it and stores the larger of the two values. Each of these operations requires a constant amount of work.

Since every entry is only processed once regardless of the input data, the best-case and worst-case runtimes are both O(nH).

After the DP table is complete, the program traces backward through keep_course to get the final list of selected courses. This step examines at most n courses, resulting in an additional O(n) runtime. However, since this step is linear, it does not change the overall runtime of the algorithm. Furthermore, the tables (dp, selected_count, and keep_course) all have (n+1)*(H+1) entries, so the total memory required grows proportionally with both the number of courses and the maximum study hours. Therefore, the space required is O(nH).

Test Cases

The following test cases from the project definition were run to confirm functionality.

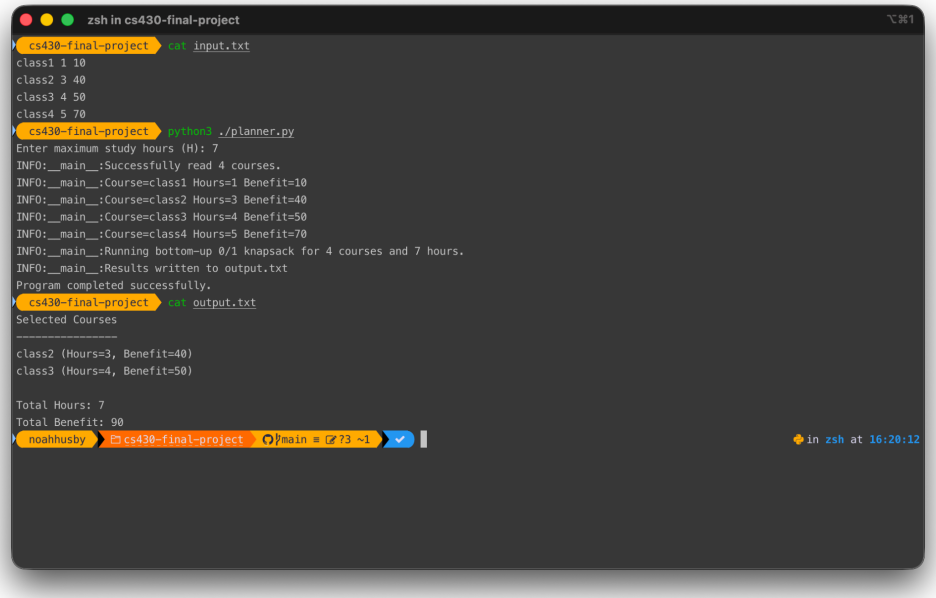

Figure 3: Test Case 1

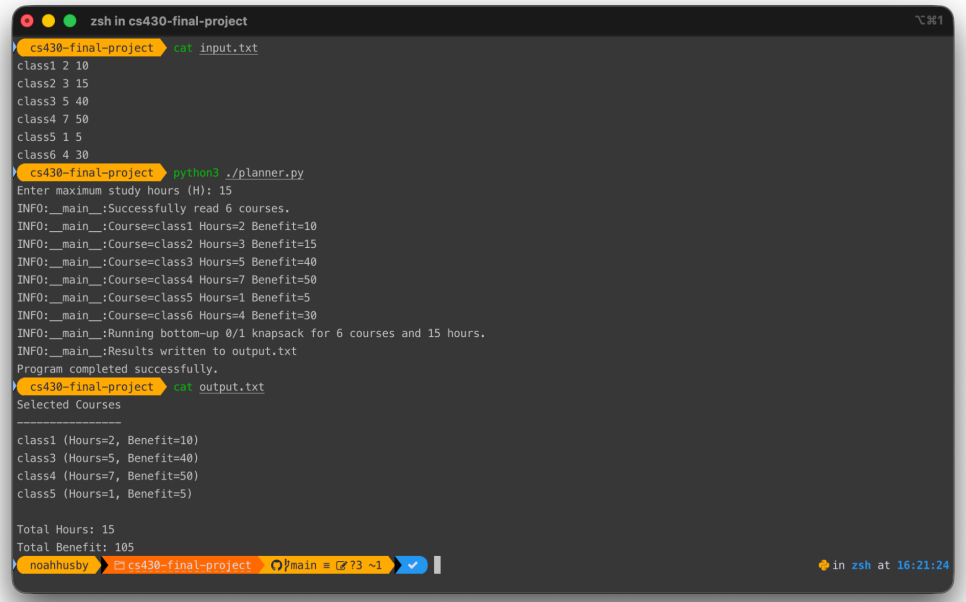

Figure 4: Test Case 2

Resources

Github Repository: https://github.com/noahhusby/knapplan
Runtime information can be found in runtime.txt OR README.md